

Análisis de Metadatos II: Unidad 3- Parte 2

Lenguaje de programación Python

Jair Pallares Castellon, MSc.



Introducción a la limpieza y estructuración de datos: Parte 2

Ficheros binarios

- Es la forma en la cual realmente se almacena la información.
- Esto puede ser almacenado en un determinado formato (ASCII, UTF-8, UTF-16...) cada vez que escribimos o leemos.
- En Los ficheros binarios no existe esa codificación trabajando los objetos de tipo *bytes* qué son secuencias de *bytes* individuales
- Con los ficheros de texto tratamos con cadenas, pero a veces nos interesa trabajar en modo binario.
 1. Si no trabajamos con datos de tipo texto. Por ejemplo, al tratar con un fichero gráfico cómo puede ser una imagen.
 2. Sí necesitamos un gran control sobre los datos de texto.

Abrir un fichero Binario: Modos de apertura, Método open()

Si queremos trabajar en modo binario debemos usar las siguientes opciones:

- **rb** Abre el fichero en modo lectura (red) binaria.
- **wb** Abre el fichero en modo escritura (write) binaria. Si no existe lo crea y si existiese lo sobrescribe.
- **ab** Abre el fichero en modo escritura (append) binaria. sí el fichero no existe lo crea. Si no añade los datos al final del fichero.
- **wb+** Abre el fichero en modo modo lectura y escritura binaria.
- **rb +** Abre el fichero en modo lectura y escritura binaria. En este modo el apuntador de ficheros se coloca al inicio de este.
- **ab+** Abre el fichero en modo lectura y escritura (añadido) binaria. Si el fichero no existe lo crea. Si existe al abrirlo el apuntador de fichero se coloca al final de este.

Leer y escribir en ficheros de binarios: métodos `read()`, `write()`, `readline()`, `readlines()`, `tell()`, `seek()` y `close()`

Los comandos son similares a los descritos para el caso de los ficheros de texto:

- **`write(misbytes: bytes): int`**

Escribe los *bytes* *misbytes* en el fichero y devuelve el número de *bytes* escritos.

- **`read(num_b: int): bytes`**

Lee *num_b* bytes desde el fichero. Si *num_b* es negativo o *None*, lee hasta el final del fichero. Si el apuntador del fichero está al final de este, *read()* devolverá en cualquier caso una cadena vacía (`""`)

- **readline(límite_b: int): bytes**

Lee y devuelve bytes hasta que aparece un cambio de línea, el *límite_b* (si lo incluimos), o el final del fichero. Si estamos ya en él, devuelve una cadena vacía (“”).

- **readlines(límite: int): bytes**

Lee y devuelve sobre las líneas del fichero. Sí incluimos el argumento *límite* leerá hasta la línea en la que esté el byte con el número indicado. Si *límite* coincide con un cambio de línea, nos devolverá también la línea siguiente. Si estamos en el final del fichero devolverá una lista vacía.

- **tell(): int**

Nos devuelve la posición numérica en la que está el AF, es decir, el número de byte al que actualmente apunta.

- **seek(desp: int, ref: int): None**

Desplaza el apuntador del fichero una serie de posiciones indicadas mediante el número entero **desp** es tomado como referencia el valor de **ref** que puede ser:

0 el inicio del fichero, por lo que en este caso desp podría ser positivo o 0.

1 el valor actual del apuntador del fichero, por lo que desp podría ser negativo, positivo o 0.

2 el final del fichero, por lo que desp será negativo o cero.

- **close(): None**

Cierra el fichero. Si estuviese ya cerrado no hace nada.

Adicional a los métodos mostrados anteriormente,

1. La opción para usar **for** con el formato:

for line in file

2. La función **list()**, con el formato:

list(nombre_lógico_fichero)

En los ficheros binarios, los métodos `tell()` y `seek()` nos permiten mayor control que los que teníamos con los ficheros de texto.